AN0194
Application Note

AT32 PMSM FOC Hall Demo(E-Bike-Scooter)

Introduction

This application note describes how to use a motor development board with vector control algorithm to drive a three-phase permanent magnet synchronous motor (brushless DC motor) with Hall sensors. It covers project environment setup, program documentation structure, macro definitions, state machine workflow, and other details. The solution is primarily applied to two-wheel control systems such as e-motorcycles, e-bikes, and e-scooters, providing features including bus current limiting and automatic MOSFET's $R_{DS(on)}$ calibration. ArteryTek offers a variety of motor development boards, and this document uses the AT-MOTOR-EVB motor development board paired with the AT32F413RCT7 sub-board as an example for explanation.

Applicable products:

Product series	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32L021 series
	AT32M412/M416 series

Contents

List of tables

List of figures

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)
Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Hall sensor position detection

Sensored FOC sine wave control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control

Auto tuning functions:

- Hall state sequence self-learning (for hall sensor only)
- Motor winding parameters identification
- Current PI controller parameters self-tuning

E-Bike-Scooter control mode:

- DC current limit
- MOS's RDS(on) auto calibration

2 Environmental requirements

2.1 Hardware requirements

A PMSM (BLDC) motor, AT-Link or third-party debugger, and motor control evaluation board (AT-MOTOR-EVB) are required. For details about hardware configurations, please refer to *AT32_LV_Motor_Control_EVB_User _Manual* (UM0011 for EVB_V1.x or UM0014 for EVB_V2.x).

- PMSM (BLDC) motor: JK42BLS01-X056ED

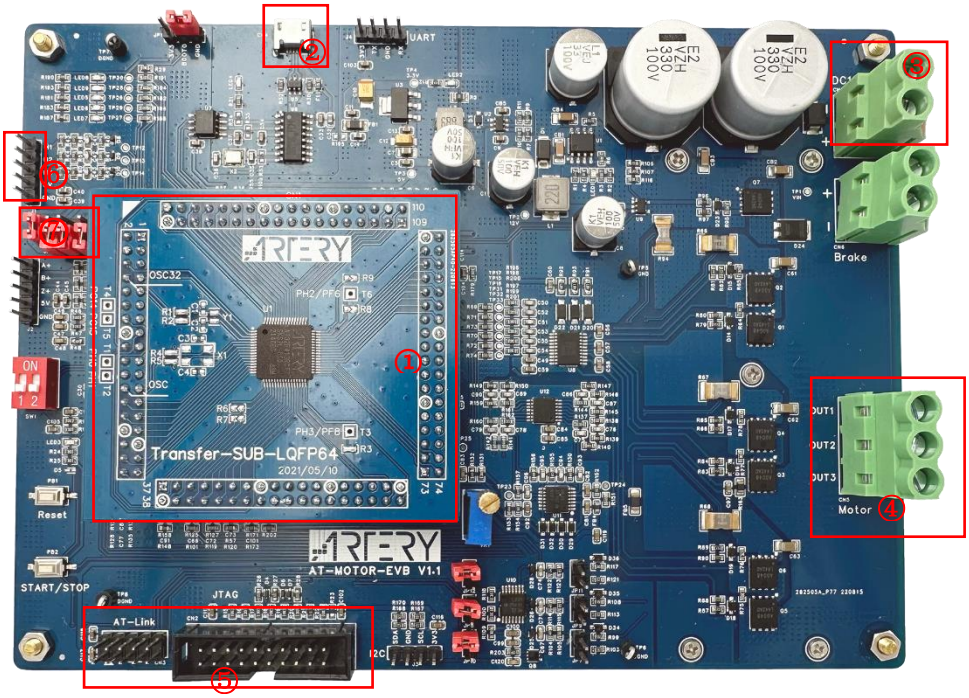
Figure 1. JK42BLS01-X056ED motor

Table 1. Motor related specifications

Item	JK42BLS01-X056ED
Number of poles	8
Rated voltage	DC 24V
Rated speed	4000 RPM
Rated torque	0.0716 N·m
No-load current	0.3 Amps
Output power	30 W
Line-to-line resistance	1.8Ω
No-load speed	6800 RPM
BEMF constant	4.36 V/Krpm
Torque constant	0.042 N·m/A
Encoder resolution	1000 P/R
Insulation grade	Class B
Winding type	Delta

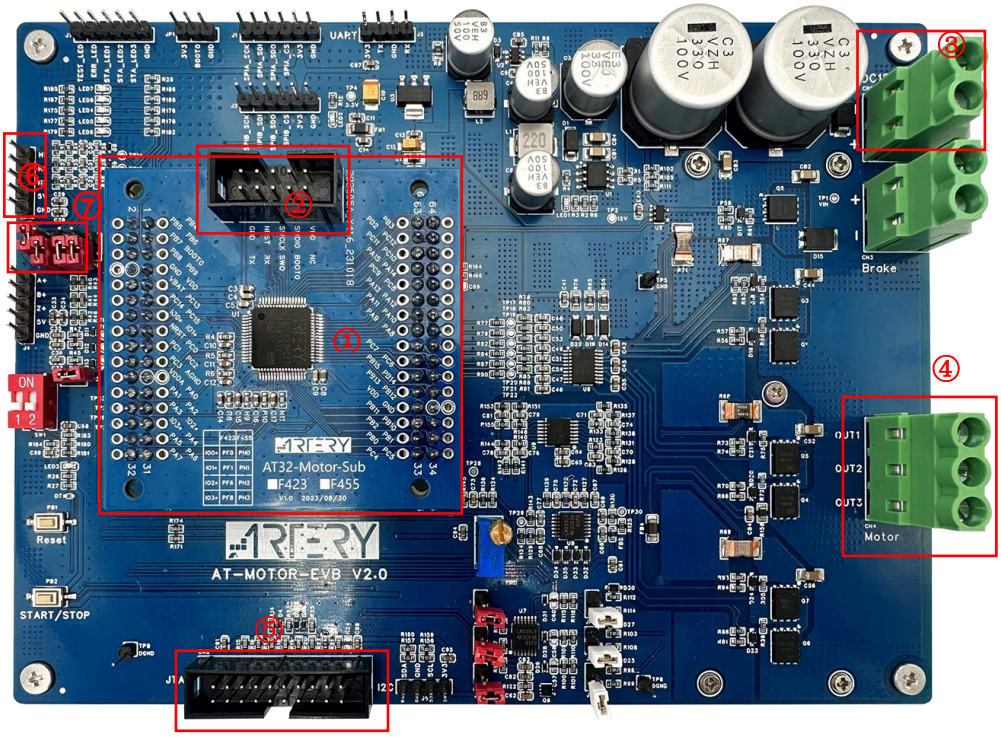
- Debugger
- Motor control evaluation board: AT-MOTOR-EVB V1.x or AT-MOTOR-EVB V2.x

Figure 2. AT-MOTOR-EVB V1.x



① AT32 MCU transfer board (part number: AT32F413RCT7)
② USB type B-to-RS-232 connector: connected to PC via USB cable, for tuning motor control parameters
③ Power supply CN1: connected to 24 V
④ Driver output connector CN5: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively
⑤ AT-Link or J-Link connector for debugging purposes
⑥ Hall sensor connector J1: connected with Hall sensor cable
⑦ Jumper settings: JP2 CLOSED; JP3, JP4, JP5, JP6 OPEN

Figure 3. AT-MOTOR-EVB V2.x



① AT32 MCU transfer board (part number: AT32F413RCT7)
② AT_Link connector: used for debugging purposes and connecting with UI for tuning motor parameters
③ Power supply CN1: connected to 24 V
④ Driver output connector CN4: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively
⑤ J-Link connector: for debugging purposes
⑥ Hall sensor connector J3: connected with Hall sensor cable
⑦ Jumper settings: JP2 CLOSED; JP3, JP4, JP5, JP6 OPEN

2.2 Software requirements

1) Taking AT32F413 as an example, open AT32F413_MC_Library_Project\motor_evb_2v0 \at32f413\pmsm_foc_hall_sensor_E-Bike-Scooter(note [1])

2) Run “ArteryMotorMonitor.exe” (installation not required)

3) For Keil IDE environment, it is necessary to modify “Read/Only Memory” in “Options” depending on AT32 MCU Flash memory size. See Table 2 for details.
Example: for a 256-KB AT32F413RCT7, its IROM1 start address is at 0x8000000, and the size is 0x3F800, while its IROM2 start address is at 0x803F800 and the size is 0x800, as shown in Figure 4.
For AT32IDE environment, it is necessary to modify ID file according to AT32 MCU Flash memory size, as shown in Figure 5.
In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 6).

Table 2. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: This demo does not support using of Keil complier version 5 and O0 optimization level for compiling purpose simultaneously. Thus either Keil compiler version 6 or O1 optimization level is used for this purpose. As AT32 BSP source code does not support V6.15 compiler, it is recommended to use Keil compiler version 5 and O1 optimization level.

Figure 4. AT32F413RCT7 ROM configuration (in Keil)

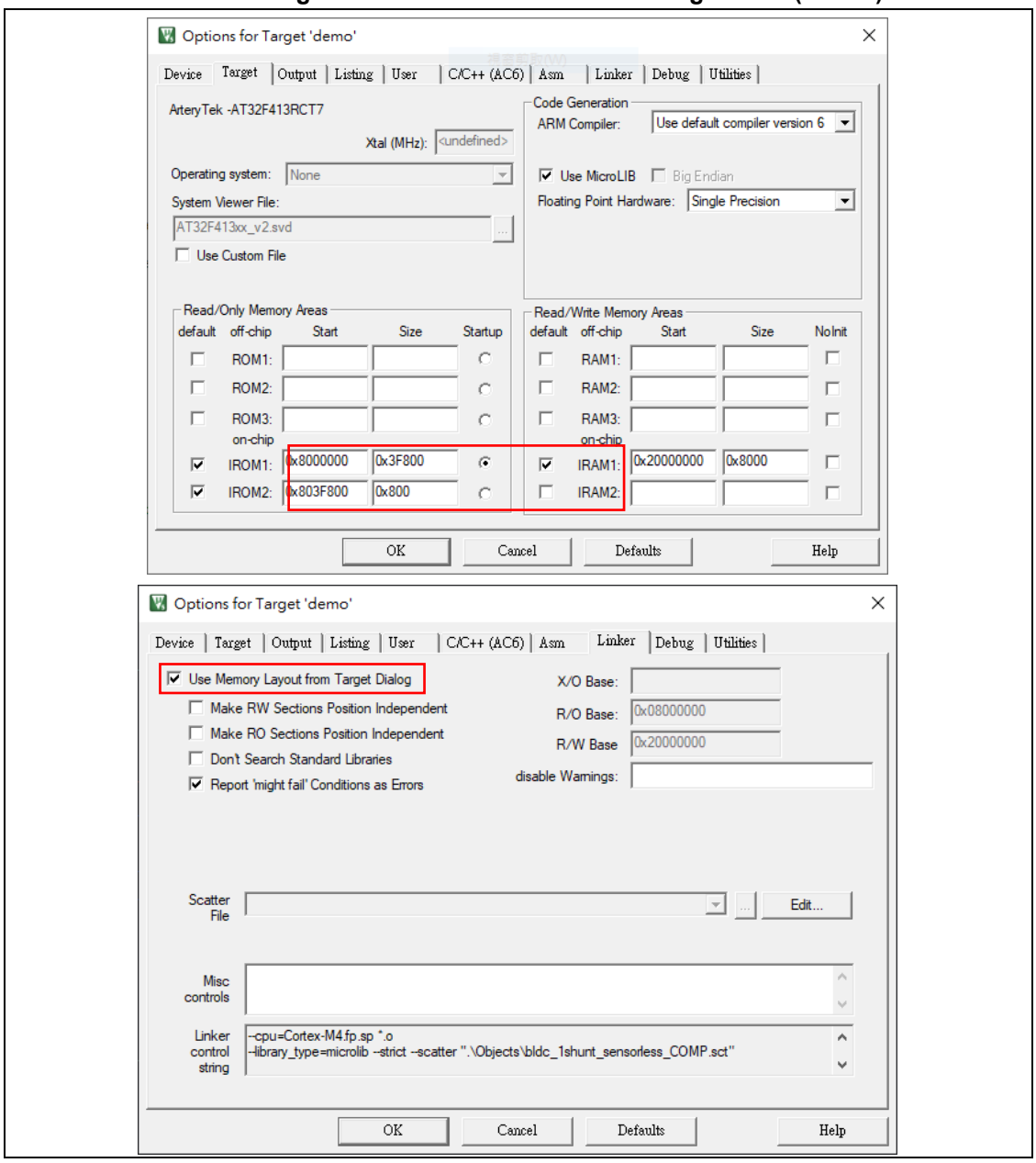
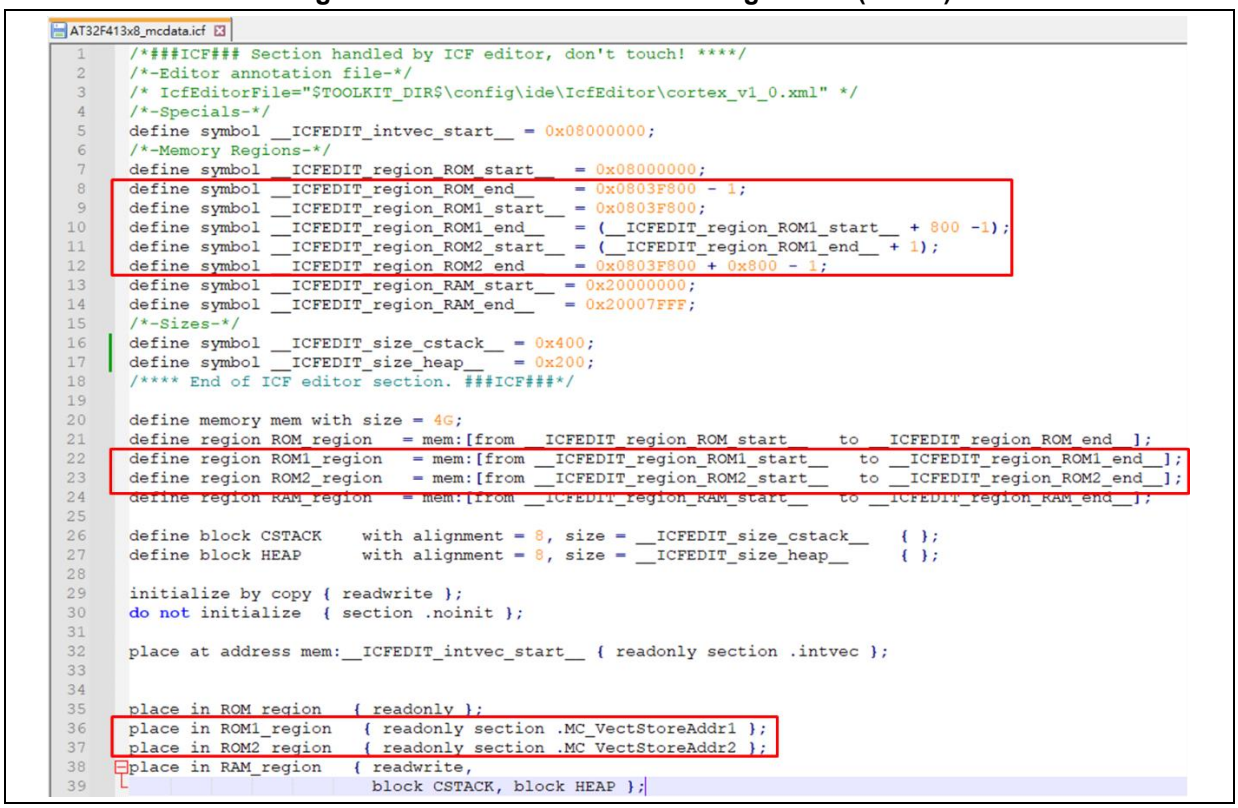


Figure 5. AT32F413RCT7 ROM configuration (in AT32 IDE)

AT32F413xC_FLASH.ld

```
_estack = 0x20008000;    /* end of RAM */

/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200;      /* required amount of heap  */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Specify the memory areas */
MEMORY
{
FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
MC_DATA (r)     : ORIGIN = 0x0803F800, LENGTH = 0x800
RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
}

/* Define output sections */
SECTIONS
{
  /* The startup code goes first into FLASH */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  .mc_data :
  {
    . = ALIGN(4);
    . = ORIGIN(MC_DATA);
    _MC_VectStoreAddr1 = .;
    *(.MC_VectStoreAddr1);
    . = ALIGN(4);
    . = ORIGIN(MC_DATA) + 800;
    _MC_VectStoreAddr2 = .;
    *(.MC_VectStoreAddr2);
    . = ALIGN(4);
  } >MC_DATA

  /* The program code and other data goes into FLASH */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)
```

Figure 6. AT32F413RCT7 ROM configuration (in IAR)



3 PMSM motor control library architecture

Figure 7 illustrates the overall structure of motor control files. The “motor_control_drive_param.h” header file is used to configure drive mode, control parameters, motor parameters and board parameters, while the “mc_hwio.h” header file is used to configure MCU peripheral parameters. All these parameters are then integrated into the “mc_foc_globals.h (mc_bldc_globals.h)” and programmed with the initial variables through the “mc_foc_globals.c (mc_bldc_globals.c)”. The “mc_hwio.c” can be used to initialize MCU peripherals.

Figure 7. Structure of motor control files

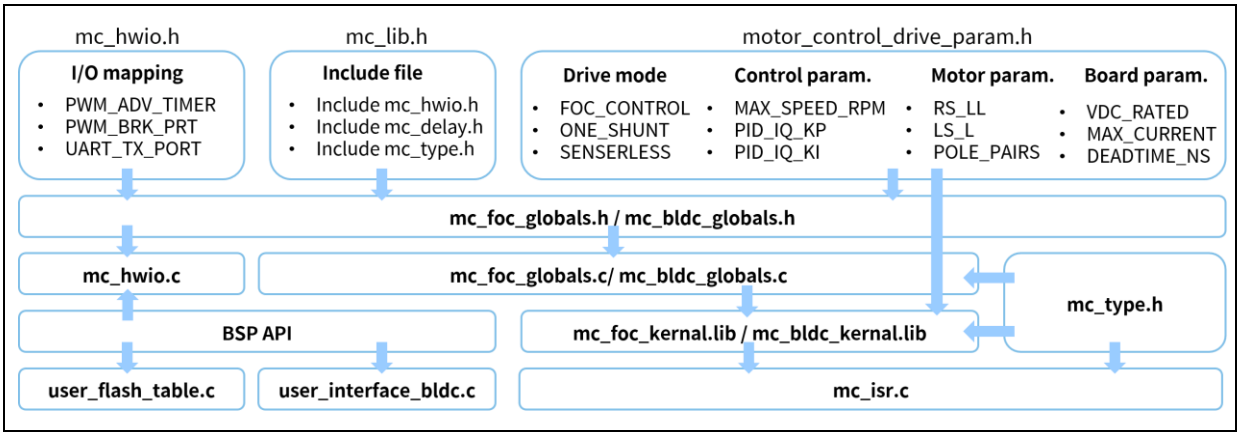


Figure 8 demonstrates the organization of files in the bldc_1shunt_sensorless demo.

The “user” is a user-defined folder containing the main programs, peripheral programs and parameter definition header files.

The “firmware” folder holds BSP files. The “mclib” folder is used to contain motor control library programs, including delay functions, communication functions, global variable setting functions and motor control library functions.

For details, refer to *AN0064 AT32_Motor_Control_Library_User_Guide* on Artery official website.

Figure 8. Structure of motor control project

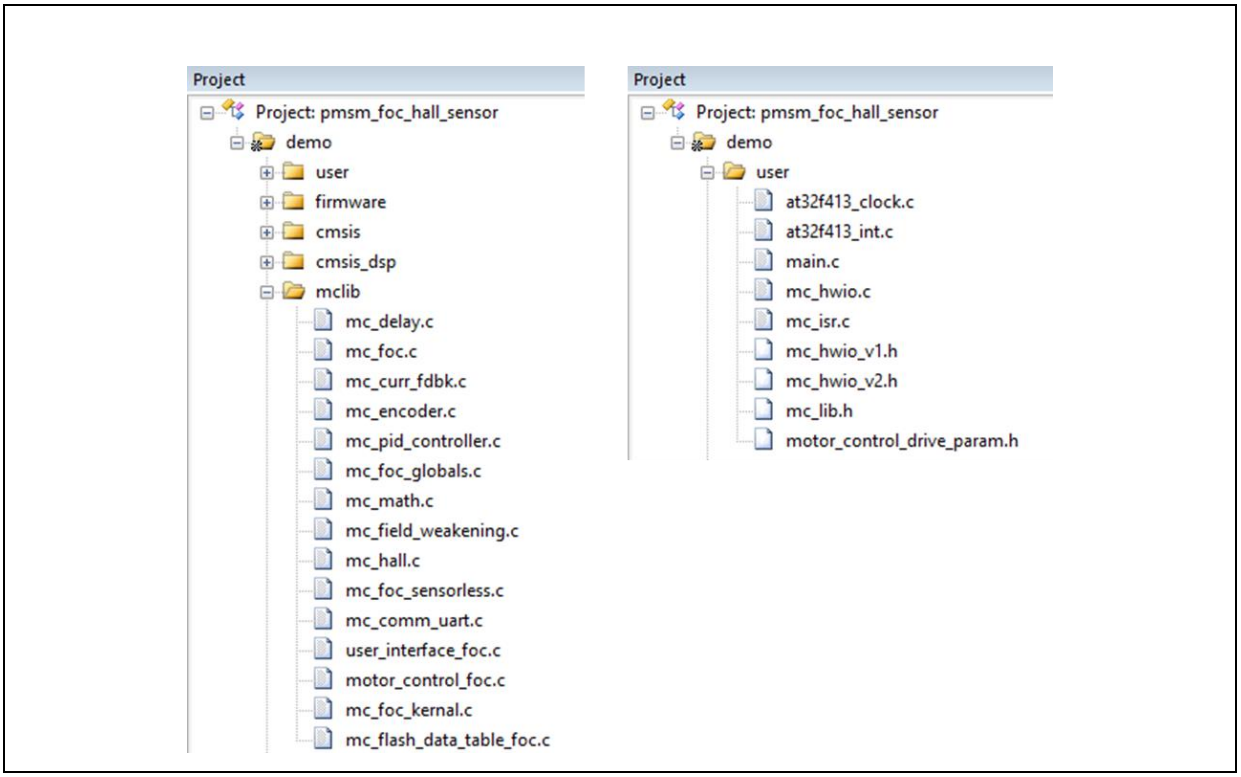


Table 3 gives a list of motor control library files.

Table 3. Motor control library file list

File name	Description
mc_lib.h	Header files
motor_control_drive_param.h	Define motor drive mode, control parameters, motor parameters, board parameters
mc_hwio.c	Hardware peripheral configuration
mc_hwio_v1.h	Hardware IO macro definition (AT-MOTOR-EVB V1.x)
mc_hwio_v2.h	Hardware IO macro definition (AT-MOTOR-EVB V2.x)
mc_isr.c	Motor control interrupt function
mc_flash_data_table_foc.c	Write flash data table
mc_flash_data_table_foc.h	Write flash data table related configuration
mc_type.h	Global variables type, enumeration definition
mc_delay.c	Delay function
mc_delay.h	Declaration of delay function
mc_comm_uart.c	Communication peripheral configuration
mc_comm_uart.h	UART communication function declaration and configuration
mc_pid_control.c	PID controller function
mc_pid_control.h	PID controller function declaration
mc_curr_fdbk.c	Current sensing function
mc_curr_fdbk.h	Current sensing function declaration
mc_math.c	Filter function

mc_math.h	filter function declaration
mc_hall.c	Hall sensor functions
mc_hall.h	Hall sensor function declaration
mc_foc_kernal.lib	Motor control library core functions (for Keil IDE, and M4F MCU with FPU)
mc_foc_kernal_noFPU.lib	Motor control library core functions (for Keil IDE, and M4 MCU without FPU)
mc_foc_kernal_m0plus.lib	Motor control library core functions (for Keil IDE, and M0+ MCU)
libmc_foc_kernal.a	Motor control library core functions (for AT32IDE, and M4F MCU with FPU)
libmc_foc_kernal_noFPU.a	Motor control library core functions (for AT32IDE, and M4 MCU without FPU)
libmc_foc_kernal_m0plus.a	Motor control library core functions (for AT32IDE, and M0+ MCU)
mc_foc_kernal.a	Motor control library core functions (for IAR system, and M4F MCU)
mc_foc_kernal_noFPU.a	Motor control library core functions (for IAR system, and M4 MCU)
mc_foc_kernal_m0plus.a	Motor control library core functions (for IAR system, and M0+ MCU)
mc_foc_kernal.h	Motor control library core function declaration
motor_control_foc.c	Motor control functions
motor_control_foc.h	Declaration of motor control functions
mc_foc.c	FOC vector control functions
mc_foc.h	Declaration of FOC vector control functions
mc_encoder.c	Encoder functions
mc_encoder.h	Declaration of encoder functions
mc_field_weakening.c	Field weakening functions
mc_field_weakening.h	Declaration of field weakening functions
mc_foc_sensorless.c	Sensorless functions
mc_foc_sensorless.h	Declaration of sensorless functions
mc_curr_decoupling.c	Current decouple control feature
mc_curr_decoupling.h	Declaration of current decouple control
MTPA_MTPV_table.c	Maximum Torque Per Ampere (MTPA)/Maximum Torque Per Volt (MTPV) field-weakening lookup table function
MTPA_MTPV_table.h	Declaration of Maximum Torque Per Ampere (MTPA)/Maximum Torque Per Volt (MTPV) field-weakening lookup table function
mc_foc_globals.c	Global variables definition and default value, global function definition
mc_foc_globals.h	Declaration of global variables and global functions, macro definitions
user_interface_foc.c	Communication interface functions
user_interface_foc.h	Declaration of communication interface functions

1) motor_control_drive_param.h file

- This file is divided into four parts and used to configure drive mode, control parameters, motor parameters and board parameters.
- Table 4 is a list of drive mode macro definitions. The user can use them to define the desired mode. This demo project uses Hall sensor to detect motor position. For current sampling, there are three methods to choose from: 3-shunt current sensing, 2-shunt current sensing, and 1-shunt current sensing. The selection of field weakening control depends on actual needs.

Table 4. Drive mode definitions

Definition name	Description
FOC_CONTROL	FOC vector control mode
THREE_SHUNT	3-shunt current sampling
TWO_SHUNT	2-shunt current sampling
ONE_SHUNT	1-shunt current sampling
U_V_SHUNT	U and V shunt (for TWO_SHUNT only)
V_W_SHUNT	V and W shunt (for TWO_SHUNT only)
U_W_SHUNT	U and W shunt (for TWO_SHUNT only)
AT_MOTOR_EVB_V1	For motor control evaluation board V1.x (AT_MOTOR_EVB_V1.x)
AT_MOTOR_EVB_V2	For motor control evaluation board V2.x (AT_MOTOR_EVB_V2.x)
GATE_DRIVER_LOW_SIDE_INVERT	Enable lower-side MOS inverted output
HALL_SENSORS	Hall sensor
FIELD_WEAKENING	Field weakening control
CURRENT_LP_FILTER	Get d/q-axis current low-pass filtering signals (vector control)
AC_CURRENT_LP_FILTER	Get A/B/C Phase Low-Pass Filtered Current Signals (Vector Control/FOC)
INTERNAL_CLOCK_SOURCE	Internal clock source
MOTOR_PARAM_IDENTIFY	Winding parameters identification
E_BIKE_SCOOTER	E-bike/scooter mode
DC_CURRENT_LIMIT	DC current limit (for E_BIKE_SCOOTER) (note [2])
RDS_AUTO_CALIBRATION	MOS RDS auto calibration (for E_BIKE_SCOOTER) (note [2])
CURRENT_DECOUPLE_CTRL	Current decouple control (NA for AT32L021)
BRAKING_RESISTOR	Use an external braking resistor
SPMSM	Surface-mounted permanent magnet synchronous motor
IPMSM	Interior Permanent Magnet Synchronous Motor
IPM_MTPA_MTPV_TABLE	MTPA/MTPV Table Generation (IPMSM-Specific) (AT32L021 series Excluded)
IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table control (IPMSM-Specific) (AT32L021 series Excluded)

USE_MOTOR_MONITOR	Artery motor control UI tool
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET On-Resistance
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (Applicable for MCUs with dual or more ADCs; Not compatible with ONE_SHUNT topology)

Note [2]: When defining the DC_CURRENT_LIMIT or RDS_AUTO_CALIBRATION by the AT-MOTOR-EVB board, a 1-shunt current sensing mode is not supported and the following components must be replaced: AT-MOTOR-EVB V1.x: R148=10kΩ, C91=0.1μF; AT-MOTOR-EVB V2.0: R154=10kΩ; AT-MOTOR-EVB V2.1: R140=10kΩ. Additionally, to use RDS_AUTO_CALIBRATION, a 3-shunt lower-side MOS FET's $R_{DS(on)}$ should be used as a 3-shunt current sampling resistor.

- Table 5 presents the motor control parameters that are used in this demo. The user can set the desired parameters according to actual needs, such as d/q-axis PI controller, speed PI controller, etc.

Table 5. Control parameter definition

Definition item	Description
PWM_FREQ	PWM output frequency (unit: Hz)
MOTOR_CONTROL_MODE	Motor control mode (speed control, torque control, etc.; see motor_control_mode in the type.h for details)
CTRL_SOURCE	Control source settings (external source control/software control)
UI_UART_BAUDRATE	Baud rate of UI communication serial port (preset value: 1.5M)
TUNE_TARGET_CURRENT	Target current value for tuning current loop PI control parameters (unit: ampere)
TUNE_CURRENT_TOTAL_PERIOD	Total period for tuning current pulse for current loop PI control parameters (unit: ms)
TUNE_CURRENT_STEP_PERIOD	Step period for tuning current pulse for current loop PI control parameters (unit: ms)
SPEED_LOOP_FREQ	Speed loop control frequency (unit: Hz)
MIN_SPEED_RPM	Minimum motor speed (unit: rpm)
MAX_SPEED_RPM	Maximum control speed (unit: rpm)
MIN_SENSE_SPEED	Minimum Detectable Speed (Unit: rpm) (For Hall Sensors Only)
STABLE_SPEED_RPM	Stable speed (unit: rpm) (for hall sensor only)
SLICK_SPEED_RPM	Slick speed (unit: rpm) (for hall sensor only)
ACC_SPD_SLOPE	Acceleration speed slope (unit: rpm/ms, systick=1kHz)
DEC_SPD_SLOPE	Deceleration speed slope (unit: rpm/ms, systick=1kHz)
SP_MAX_VOLT	Maximum voltage of external command source (unit: voltage)
SP_THRESHOLD	Threshold voltage of external command source (unit: voltage)
SP_RUN_VALUE	Lowest voltage to startup in external source mode (unit: voltage)
SP_STOP_VALUE	Threshold voltage to stop in external source mode (unit: voltage)
PID_SPD_KP_DIV	Speed controller proportional gain divisor (Q16 mode)
PID_SPD_KI_DIV	Speed controller integral gain divisor (Q16 mode)
PID_SPD_KP_DEFUALT	Speed controller proportional gain (Q15 mode)
PID_SPD_KI_DEFUALT	Speed controller integral gain (Q15 mode)
PID_ID_KP_DEFUALT	d-axis current proportional gain (Q15 mode)
PID_ID_KI_DEFUALT	d-axis current integral gain (Q15 mode)
PID_ID_KP_GAIN_DIV	d-axis current proportional gain divisor (Q16 mode)
PID_ID_KI_GAIN_DIV	d-axis current integral gain divisor (Q16 mode)
PID_IQ_KP_DEFUALT	q-axis current proportional gain (Q15 mode)
PID_IQ_KI_DEFUALT	q-axis current integral gain (Q15 mode)

Definition item	Description
PID_IQ_KP_GAIN_DIV	q-axis current integral gain divisor (Q16 mode)
PID_IQ_KI_GAIN_DIV	q-axis current integral gain divisor (Q16 mode)
AUTO_TUNE_CURR_BANDWIDTH	Current PI controller bandwidth (used for self-tuning current PI controller parameters)
CURR_LP_BANDWIDTH	Current low-pass filter bandwidth (unit: Hz)
AC_CURR_LP_BANDWIDTH	a/b/c phase current low-pass filter band width (unit: Hz)
OLC_ANGLE_INC	Open loop angle increment (per PWM output frequency)
OLC_VOLT	Open loop voltage output (unit: voltage)
LEARN_OLC_VOLT	Open loop voltage in HALL self-learning mode (unit: voltage)
LEARN_OLC_ANGLE_INC	Angular increment in HALL self-learning mode (per PWM output frequency)
LEARN_TIME	Hall self-learning time (unit: ms)
LEARN_ALIGN_TIME	Hall self-learning align time (unit: ms)
FW_MAX_ID_CURR	Maximum d-axis current for field weakening control (unit: ampere)
FW_KP_GAIN	Field weakening control proportional gain (Q15 mode)
FW_KI_GAIN	Field weakening control integral gain (Q15 mode)
FW_KP_GAIN_DIV	Field weakening control proportional gain divisor (Q16 mode)
FW_KI_GAIN_DIV	Field weakening control integral gain divisor (Q16 mode)
REVERSE_MAX_SPEED_RPM	Maximum reverse speed (unit: rpm) (for E_BIKE_SCOOTER only)
REVERSE_CURRENT	Reverse current command (unit: ampere) (for E_BIKE_SCOOTER only)
BRAKING_CURRENT	Brake current command (unit: ampere) (for E_BIKE_SCOOTER only) (note[3])
MAX_LOCK_CURRENT	Maximum Parking/Theft-Deterrent Current Command (unit: ampere) (Dedicated for E_BIKE_SCOOTER)
ANTI_THEFT_INIT_CURRENT	Anti-theft Initial Current Value (unit: ampere) (E-BIKE_SCOOTER only)
ANTI_THEFT_INC_CURRENT	Anti-theft Current Increment per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
ANTI_THEFT_DEC_CURRENT	Anti-theft Current Decrement per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
PARKING_LOCK_INIT_CURRENT	Parking Initial Current Value (unit: ampere) (E-BIKE_SCOOTER only)
PARKING_LOCK_INC_CURRENT	Parking Current Increment per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
PARKING_LOCK_DEC_CURRENT	Parking Current Decrement per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
ANTI_THEFT_LOCK_TIME	Anti-theft Maximum Current Lock Release Time (unit: ms) (E-BIKE_SCOOTER only)

Definition item	Description
PARKING_LOCK_TIME	Parking Maximum Current Lock Release Time (unit: ms) (E-BIKE_SCOOTER only)
HIGH_PWM_DUTY_CYCLE	D-axis voltage command during low-speed voltage control (unit: voltage) (Exclusive to LOW_SPEED_VOLT_CTRL Mode)
MAX_DUTY_PCT	Dynamic Adjustment of Current Sampling Position at High PWM Duty Cycle (TWO_ADC_CONVERTERS only)

Note [3]: If braking feature is not needed, the braking_current must be set to 0. By doing so, the motor will naturally slow down without braking function.

- Table 6 presents the driver parameters that are used in this demo, including dead time, current sensing resistance, and current amplifier gain. Note that the corresponding parameters should be modified if a different motor drive board is to be used.

Table 6. Driver parameters definition

Definition item	Description
VDC_RATED	DC-BUS voltage (unit: voltage)
BAT_LOW_VOLT	Lowest battery voltage (unit: voltage)
V_SENSE_GAIN	Voltage feedback ratio (refer to Section 5.5 in *UM0011_AT32_LV_Motor_Control_EVB_User_Manual*)
ADC_REFERENCE_VOLT	ADC reference voltage (unit: voltage)
ADC_DIGITAL_SCALE_12BITS	ADC resolution
SYSTEM_CORE_CLOCK	System frequency (unit: Hz)
TMR_CLK	Timer clock frequency (unit: Hz)
DEADTIME_CLK_SFT_BITS	Shift bit number of frequency divider for dead time generator
DEADTIME_NS	Dead time (unit: ns)
MIN_INTERVAL_TIME	Minimal interval time between two phase signals when PWM shifts (unit: ns)
ADC_TRIG_DELAY_TIME	Current sampling delay time (unit: us)
ADC_TRIG_LAG_TIME	ADC trigger lag time for current sensing (Two-Phase Low-Level Condition) (Unit: ns) (only for TWO_ADC_CONVERTERS && THREE_SHUNT)
SW_OP_INP_MODE_LEAD_TIME	OP Input Mode Switching Lead Time Prior to ADC Trigger (for chips with internal OP like M412)
MAX_CURRENT	Maximum current (unit: ampere)
MIN_CURRENT	Minimum current (unit: ampere)
DC_MAX_CURRENT	Maximum DC current (unit: ampere) (for DC_CURRENT_LIMIT only)
CURRENT_SPAN_SHIFT	Current span shift
R_SHUNT	Shunt resistance (unit: Ω)
OP_GAIN	Current amplifier gain (refer to Section 5.3.2 in

Definition item	Description
	UM0011_AT32_LV_Motor_Control_EVB_User_Manual)
CURR_OFFSET_VOLT	Zero current offset (refer to Section 5.3.2 in *UM0011_AT32_LV_Motor_Control_EVB_User_Manual*) (unit: voltage)
RDC_SHUNT	RDC shunt (unit: Ω) (for DC_CURRENT_LIMIT only)
DC_OP_GAIN	DC current OP gain (for DC_CURRENT_LIMIT only)
IDC_OFFSET_VOLT	DC zero current shift (unit: voltage) (for DC_CURRENT_LIMIT only)
OVER_CURRENT_SW	Software Overcurrent Threshold (unit: ampere)
DAC_VREF_SOURCE	DAC Source Selection for Internal OP Overcurrent Protection (Dedicated to chips with internal OP, e.g., M412 series)
OCP_CURRENT	Hardware Overcurrent Threshold (unit: ampere)
BUS_CURR_CMP_OCP_VOLT	Internal OP Overcurrent Protection Voltage (unit: volt) (Dedicated to chips with internal operational amplifier, e.g., M412 series)
TMR_BRK_FILTER_COUNT	Overcurrent Protection Break Input Filter Count (Dedicated to chips with break input function, e.g., M412 series)
OVER_VOLT_THRESHOLD	Over-voltage threshold point (unit: voltage)
UNDER_VOLT_THRESHOLD	Under-voltage threshold point (unit: voltage)
V0_V	Parameter V0 of the approximate curve of NTC temperature vs. voltage (Note [4])
T0_C	Parameter T0 of the approximate curve of NTC temperature vs. voltage (Note [4])
dV_dT	Parameter dV/dT of the approximate curve of NTC temperature vs. voltage (Note [4])
OVER_TEMP_THRESHOLD	Over-temperature threshold point (unit: Celsius degrees)
MC_ERROR_MASK	Error detection mask

Note [4]: Approximate curve equation of voltage vs. temperature is V[V]=V0+dV/dT[V/Celsius](T-T0)[Celsius].*

- Table 7 presents the motor parameters used in this demo, including number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 7. Motor parameter definition

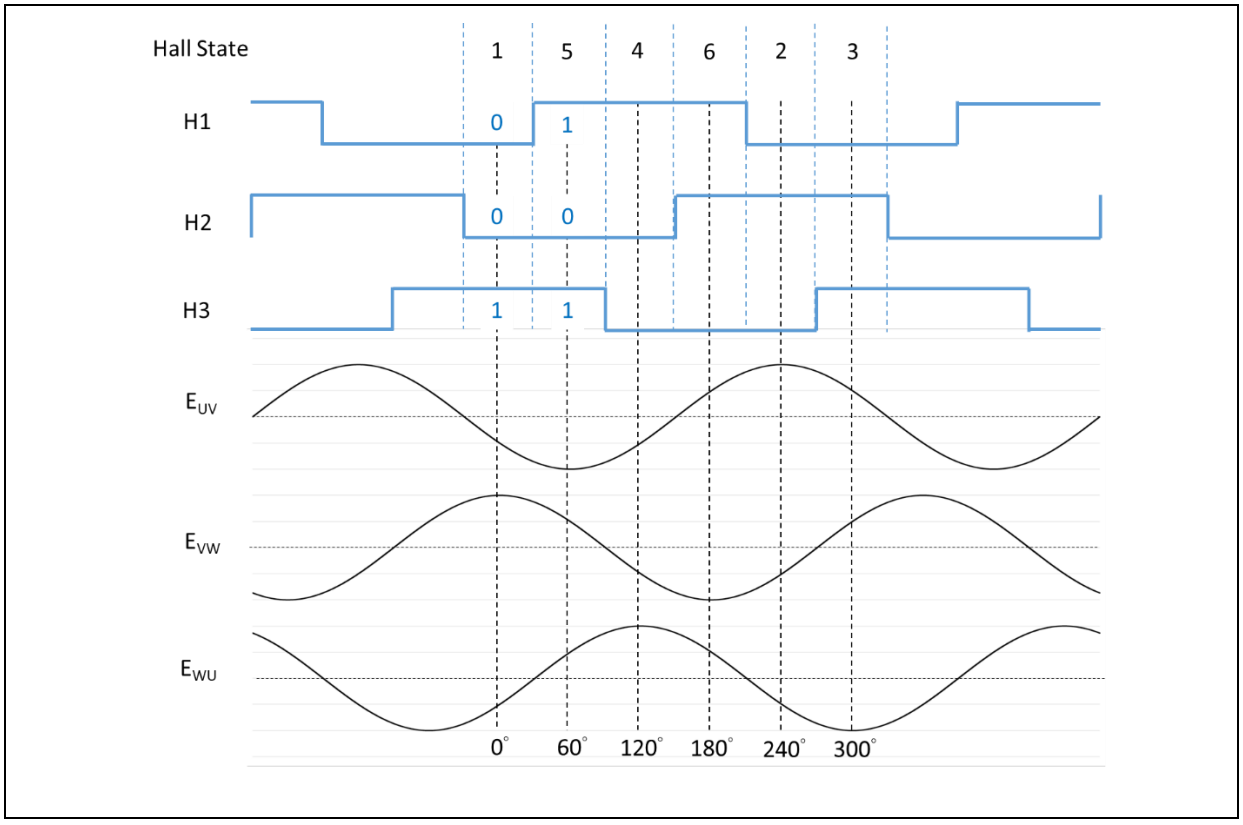
Definition name	Description
POLE_PAIRS	Number of pole-pairs
KE	Back EMF constant (KE) (unit: V/rpm)
LD_LQ_RATIO	Ld to Lq ratio
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor rotation direction after Hall sensor self-learning, note[3]
HALL_LEARN_0_STATE	Hall state at 0 degree electrical angle after Hall self-learning (note[3])

Definition name	Description
HALL_LEARN_1_STATE	Hall state at 60 degree electrical angle after Hall self-learning (note[3])
HALL_LEARN_2_STATE	Hall state at 120 degree electrical angle after Hall self-learning (note[3])
HALL_LEARN_3_STATE	Hall state at 180 degree electrical angle after Hall self-learning (note[3])
HALL_LEARN_4_STATE	Hall state at 240 degree electrical angle after Hall self-learning (note[3])
HALL_LEARN_5_STATE	Hall state at 300 degree electrical angle after Hall self-learning (note[3])

Note [5]: Hall state sequence and corresponding electrical angle are configured based on BEMF and hall state.

Figure 9 presents the relationship between BLDC(JK42BLS01-X056ED) BEMF, Hall state and electrical angle. The line-to-line BEMF for three-phase motor should be in consistent with this diagram. The zero degree of angle corresponds to the maximum BEMF between VW lines, while 60-degree angle corresponds to the minimum BEMF between UV lines, and so on. Different hall state values indicate different electrical angles. However, there is no need for AT32 MCU users to spend time finding this relationship as AT32 motor control library comes with a Hall phase sequence self-learning feature capable of providing corresponding Hall state values. This is achieved by writing Hall state values after Hall phase sequence self-learning routine into the corresponding hall sensor macro definitions. See *AN0219_AT32_PMSM_FOC_Hall (E-Bike-Scooter)* for details.

Figure 9. Relationship between PMSM BEMF, Hall state and electrical angle (HALL_LEARN_DIR=0)



2) mc_hwio.h file

- This file is used to configure hardware IOs and peripherals. It also contains the declaration of the *mc_hwio.c* files. Table 8 presents the configuration of peripherals (ADC, TMR, USART and EXINT, etc.), interrupt functions and DMA channels that are used in this demo.

Table 8. Configuration of peripherals (AT32F413RCT7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command, 3-phase BEMF voltage)
ADC1	ADC1_2_IRQn	N/A	ADC1 preemptive channel conversion (bus current or phase current)
TMR1	TMR1_OVF_TMR10_IRQn	N/A	FOC control interrupt
TMR1	TMR1_CH_IRQHandler	N/A	Disable BEMF pull-up resistor GPIO interrupt (ARTERY patent)
TMR1_CH4	N/A	DMA1_CH4	Used to configure sampling time point in one-shunt mode
TMR3	TMR3_GLOBAL_IRQn	N/A	Capture hall sensor signals
TMR11	TMR1_TRG_HALL_TMR11_IRQHandle	N/A	Speed control loop interrupt
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	N/A	DMA1_CH3	USART1 RX data
EXINT	EXINT15_10_IRQn	N/A	External signals to activate the HALL self-learning interrupt, START/STOP button (start/stop motor)

3) mc_hwio.c file

- This file configuration depends on users' hardware peripherals, such as TMR, ADC, DMA and GPIO. It also contains functions such as button, LED, DIP switch. See Table 9 for details.

Table 9. Peripheral configuration functions

Function	Description
nvic_config	Interrupt priority configuration
tmr_pwm_init	PWM timer, crm clock, GPIO and DMA configuration
hall_timer_init	Configure hall sensor-related timer, crm clock and GPIO
adc_ordinary_config	ADC ordinary channel ADC, DMA and GPIO configuration
adc_preempt_config	ADC preemptive channel ADC, DMA and GPIO configuration
speed_timer_init	Configure speed control-related timer, and crm clock
uart_init	UART crm clock, GPIO and UART configuration
button_exint_init	Button interrupt event EXINT configuration
led_config	LED GPIO initialization and crm clock configuration
led_on	LED ON
led_off	LED OFF
led_toggle	LED toggles (from ON to OFF, or from OFF to ON)
led_blink	LED blinks

mode_switch_init	Switch button configuration
motor_parameter_ID_config	TMR, ADC configuration in winding parameter identification
get_int_vref_cal_ratio	Detect VDDA reference voltage ratio, used for ADC value calibration
hwdiv_config	Hardware divider configuration (for AT32L021 only)
opa_init	Busbar Current Amplification OP Configuration (AT32M412 and other chips with internal OP only)
opa_calibration	Busbar Current Amplification OP Calibration (AT32M412 and other chips with internal OP only)
ocp_cmp_config	Busbar Current Protection CMP Configuration (AT32M412 and other chips with internal CMP only)

4) mc_isr.c file

- This file contains interrupt handler functions for ADC, TMR and SYSTICK interrupts, etc. Refer to Table 10 for details.

Table 10. Motor control interrupt functions (AT32F413RCT7 as an example)

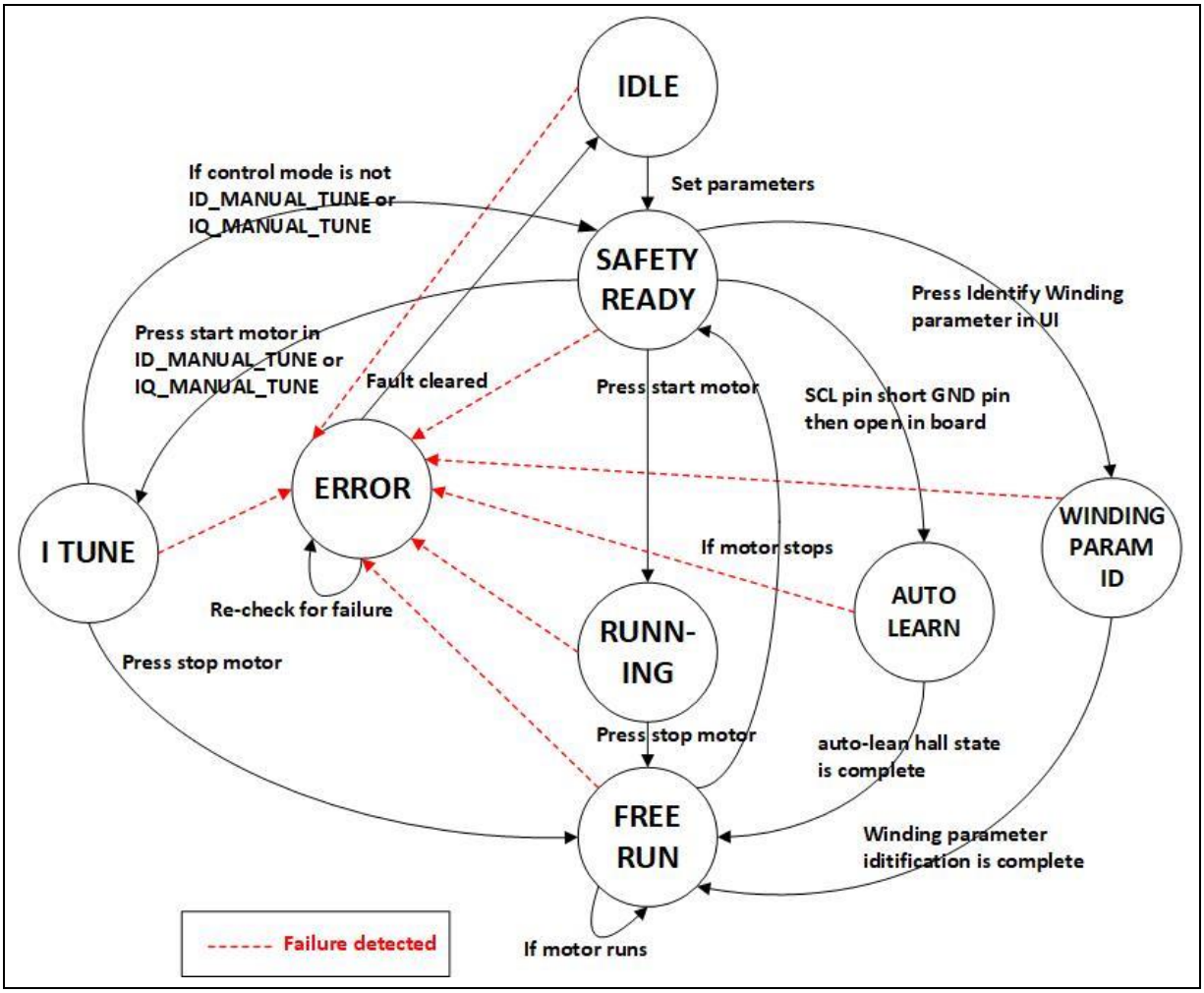
Function	Description
ADVTMR_PWM_CYCLE_IRQ (TMR1_OVF_TMR10_IRQHandler)	TIMER1 update interrupt function: rotor angle detection, vector control, open loop control, voltage loop control, current loop control
ADVTMR_PWM_BRK_IRQ (TMR1_BRK_TMR9_IRQHandler)	TMR1 brake input interrupt (PWM output disable)
ADC_SHUNT_SAMP_READY_IRQ (ADC1_2_IRQHandler)	ADC1 interrupt function: Current sampling complete
HALL_CAPTURE_IRQ (TMR3_GLOBAL_IRQHandler)	Capture interrupt function: hall signals input capture and speed capture
SPEED_LOOP_TIMER_IRQ (TMR1_TRG_HALL_TMR11_IRQHandler)	Speed control loop interrupt function
SysTick_Handler	System interrupt function (1ms): includes state machine program
BUTTON_EXINT_IRQ (EXINT15_10_IRQHandler)	External IO interrupt function: external signals to activate the Hall self-learning interrupt, START/STOP button (start/stop motor)

4 Motor control state machine

4.1 Introduction

Figure 10 illustrates the flow chart of the state machine. The initial settings of each state are checked by the Main program on a circular basis. Upon state change, the new state is programmed with initial settings. In addition, the Systick Handler interrupt function is responsible for real-time monitoring the state machine at a minisecond rate. Once failure is detected, the state machine enters “Error” state to halt motor so as to avoid potential damage to motor or driver. The motor is being stalled until the failure is cleared. The E-Bike-Scooter demo offers two modes – torque mode and non-torque mode, to showcase the state machine process. As the process, start condition and end condition are fully different in these two modes, we offer two types of state flow charts here, as shown in Figure 10 and Figure 11. For the conditions to switch between states, see Table 11 for details.

Figure 10. State machine flow chart (specific mode)



The entire flow chart consists of Idle, Safety ready, Running, Braking, Reverse run, Free run, I_tune and Error states.

Idle

The initial state of a state machine. Motor stays idle in this state. And the state machine returns to “Idle” state when a failure is cleared or motor stops running.

Safety ready

This state indicates that the motor can start up safely as all parameters are already set and the current offset value is available.

Running

This state indicates that the motor is running, and the user can adjust parameters (target speed, target current) or stop motor via UI interface.

Braking

The motor is in brake state in this mode. This mode is enabled in one of the following three situations:

1) Set the brake_flag when the motor runs normally;
2) Set current command =0 when the motor runs normally in torque mode;
3) Set speed command =0 when the motor runs normally in speed control mode.

In addition, the BRAKING_CURRENT is used to set a brake current command (when the braking mode is not used, the BRAKING_CURRENT can be set to 0, and the motor will slow down naturally). When the speed is slower than the STABLE_SPEED_RPM, the “Free run” mode is entered, stopping driver output until it reduces to 0.

Reverse run

In this mode, the motor direction is reversed. This mode is enabled the moment the users set the reverse_flag and clear the brake_flag when the motor remains in static state in torque mode or speed control mode. The REVERSE_MAX_SPEED_RPM and the REVERSE_CURRENT are used to set the maximum reverse speed and reverse current, respectively.

Free run

This state is similar to Stop mode. In this state, the driver stops output, and the motor speed will slowly reduce to zero. This state is maintained until the motor stops running. After a full stop of motor, the state machine then returns to “Safety ready” state.

Anti-theft

This state is Anti-theft mode, once triggered, the motor will lock, preventing the wheel from rotating or the motor from starting. The E-BIKE cannot be pushed or ridden normally.

Parking Lock

This state is Parking mode, manually activated after the E-BIKE comes to a complete stop, it locks the motor to prevent the E-BIKE from rolling away.

I_tune

This state is used to tune PID controller parameters. In this state, the user can tune the target current, current loop KP and KI values via UI software to generate a step current.

Winding param Id

This indicates that a motor is in winding parameters identification state. This feature is enabled by users via UI interface. Winding parameter identification is usually necessary for adjusting a new motor. The winding parameters identified is useful for sensorless vector control and tuning current PI control parameters.

Auto learn

This refer to self-learning state. Our motor control library supports Hall state sequence self-learning function, which can be applied to both six-step square wave and vector control modes.

Error

The state machine jumps to “Error” state in case of any failures.

Table 11 presents the initial state, end state and change conditions of each state when the motor runs in specific control mode.

Table 11. State switch conditions (in specific control mode)

Initial state	End state	State switch condition
IDLE	SAFETY REDAY	The control source is from UI tool, and initial parameters are already set.
		The control source is from the external circuit. The POTENTIOMETER value is less than the default value, and initial parameters are already set.
SAFETY REDAY	RUNNING	When the control mode is not IQ_MANUAL_TUNE or IQ_MANUAL_TUN, press START MOTOR or START/STOP button on the motor control board.
	I TUNE	When the control mode is IQ_MANUAL_TUNE or IQ_MANUAL_TUNE, press START MOTOR (UI tool) or START/STOP button on the motor control board.
	WINDING_PARAM_ID	Press “Identify Winding parameter” button on the UI tool
	AUTO_LEARN	Press “Hall phase sequence self-learning” button on the UI tool, or the SCL pin of the I^2C interface (AT-MOTOR-EVB V1.x:J3; AT-MOTOR-EVB V2.x:J5) on the motor control board is first shorted to GND and then open.
RUNNING	FREE_RUN	Click STOP MOTOR (UI tool) or START/STOP on the motor control board.
FREE_RUN	SAFETY REDAY	Motor stops.
	FREE_RUN	Motor runs.
I TUNE	SAFETY REDAY	When the control mode is not the ID_MANUAL_TUNE or IQ_MANUAL_TUNE
	FREE_RUN	Click STOP MOTOR (UI tool) or START/STOP on the motor control board.
WINDING_PARAM_ID	FREE_RUN	Winding parameter identification operation is complete
AUTO_LEARN	FREE_RUN	Hall state sequence self-learning operation is complete
ERROR	IDLE	Failure is cleared.

Figure 11 presents the state machine flowchart in torque/speed control mode. For the conditions to switch between states, see Table 12 for details.

Figure 11. State machine flow chart (in torque/speed control mode)

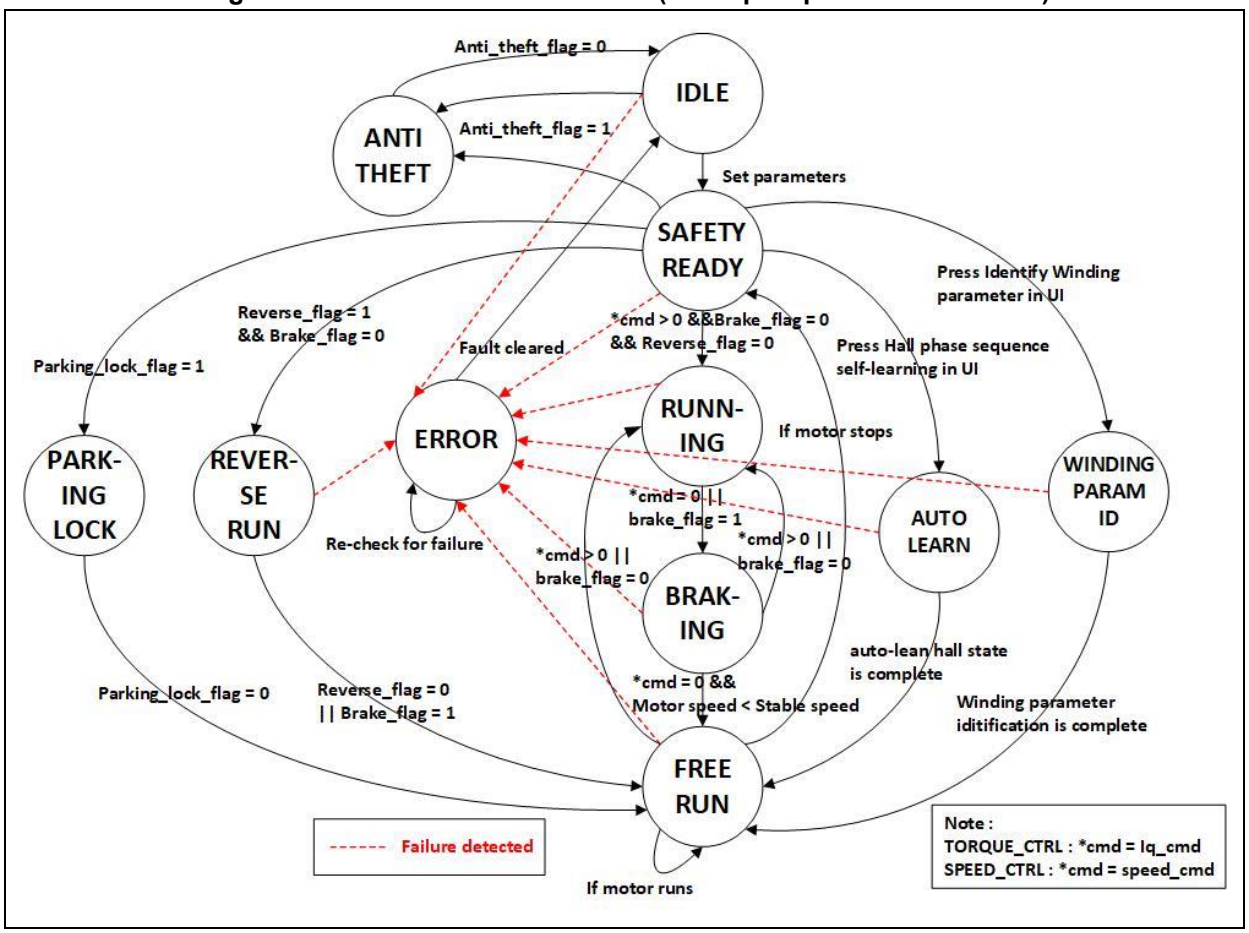


Table 12. Initial state and change conditions (in torque/speed control mode)

Start state	End state	Change conditions
IDLE	SAFETY REDAY	Control source: UI tool Initial parameters (initialization) are finished.
		Control source: motor control board POTENTIOMETER command is lower than its default value and initial parameters (initialization) are finished.
	ANTI_THEFT	When the control mode is TORQUE_CTRL and anti_theft_flag is set
SAFETY REDAY	RUNNING	When the control source is the UI tool and the control mode is TORQUE_CTRL, the Torque reference (current command) >0, or Speed reference (speed command) >0 in speed control mode (SPEED_CTRL), and reverse_flag=0, brake_flag=0, anti_theft_flag=0 and parking_lock_flag=0.
		When the control source is motor control board and the control mode is TORQUE_CTRL or SPEED_CTRL, the POTENTIOMETER command is greater than its default value and the “1” and “2” of SW1 is OFF.
	REVERSE RUN	When the UI tool is used as control source, and TORQUE_CTR or SPEED_CTRL as control mode, and the reverse_flag is set, brake_flag, anti_theft_flag and parking_lock_flag are cleared.
		When the motor control board is used as control source, the

		TORQUE_CTRL or SPEED_CTRL as control mode, and the “1” of SW1 is ON and “2” of SW1 is OFF.
	ANTI_THEFT	When the control mode is TORQUE_CTRL and anti_theft_flag is set
	PARKING_LOCK	When the control mode is TORQUE_CTRL, anti_theft_flag is cleared and parking_lock_flag is set
	WINDING_PARAM_ID	Press “Identify Winding parameter” button.
	AUTO_LEARN	Press “Hall phase sequence self-learning” button, or the SCL pin of the I²C interface (AT-MOTOR-EVB V1.x:J3; AT-MOTOR-EVB V2.x:J5) on the motor control board is first shorted to GND and then open shorted
RUNNING	BRAKING	When the UI tool is used as a control source, the TORQUE_CTRL as control mode, and the current command=0; or the SPEED_CTRL as control mode, and speed command=0 or brake_flag is set.
		When the motor control board is used as control source, and the TORQUE_CTRL or SPEED_CTRL as control mode, the POTENTIOMETER is less than its default value or the “2” of SW1 is ON.
BRAKING	FREE_RUN	When the UI tool is used as control source, the TORQUE_CTRL as control mode and current command=0, or the SPEED_CTRL as control mode and speed command=0, and the speed is less than the STABLE_SPEED_RPM value.
		When the motor control board is used as control source, the TORQUE_CTRL or SPEED_CTRL as control mode, the POTENTIOMETER is less than its default value, and the speed less than the STABLE_SPEED_RPM value.
	RUNNING	When the UI tool is used as control source, the TORQUE_CTRL as control mode and current command >0, or the SPEED_CTRL as control mode and speed command >0 and brake_flag is cleared.
		When the motor control board is used as control source, the TORQUE_CTRL or SPEED_CTRL as control mode, the POTENTIOMETER is greater than its default value and “2” of SW1 is OFF.
FREE_RUN	SAFETY REDAY	When the motor stops
	FREE_RUN	When the motor is running
	RUNNING	When the UI tool is used as control source, the TORQUE_CTRL as control mode and current command >0, or the SPEED_CTRL as control mode and speed command >0 and brake_flag is cleared.
		When the motor control board is used as control source, the TORQUE_CTRL or SPEED_CTRL as control mode, and POTENTIOMETER is greater than its default value and “2” of SW1 is OFF.
REVERSE RUN	FREE_RUN	When the UI tool is used as control source, the TORQUE_CTRL or SPEED_CTRL as control mode, and reverse_flag = 0 or brake_flag =1.
		When the motor control board is used as control source, the TORQUE_CTRL or SPEED_CTRL as control mode, and the “1” of SW1 is OFF or “2” of SW1 is ON.
ANTI_THEFT	IDLE	When the anti_theft_flag is cleared
PARKING_LOCK	FREE_RUN	When the parking_lock_flag is cleared

WINDING_PARAM_ID	FREE_RUN	Winding parameter identification operation is complete
AUTO_LEARN	FREE_RUN	Hall state self-learning operation is complete
ERROR	IDLE	Error has been cleared

5 How to use demo project

For details, refer to *AN0219 AT32 PMSM FOC Hall(E-Bike-Scooter)*.

6 Revision history

Table 13. Document revision history

Date	Version	Revision note
2023.10.03	2.0.0	Initial release.
2024.02.27	2.0.1	Added descriptions about AT-MOTOR-EVB-V2.0, winding parameter identification, auto-tune current PI controller parameters, hall sensor self-learning.
2024.05.24	2.1.0	Added descriptions regarding brake recharging mode, reverse mode and maximum speed limit in current loop control mode
2024.12.27	2.1.1	Added applicable products, including AT32F435/437 and AT32L021; Added IAR IDE and the corresponding Flash ROM configuration; Updated some definitions and description.
2025.04.11	2.1.2	Added Support for AT32F425 series and MTPA/MTPV-related macro definitions Updated description of state machine and flow chart (added speed control feature).
2025.10.09	2.1.3	Add parking/anti-theft mode, add supported models, correct flash configuration table, parameters, and state machine updates

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.